PROJEKAT R.A.D. · RECON AERIAL DRONE

Pregled koda kontrolne stanice

Backend (C++, DroneBackend) i frontend (Python, DroneCockpitUI): moduli, metode, algoritmi, niti i tok podataka, sa programerskog stanovišta.

Autor projekta: Aleksandar Lipovčić, Elektrotehnički fakultet Banja Luka

Stanje koda: grane `master` i `feature/RadioLink`, commit `2c72f14` (4. 11. 2026)

Datum dokumenta: 5. 11. 2026

Sve vrijednosti konstanti, nazivi metoda i redoslijedi koraka u ovom dokumentu provjereni su u izvornom kodu tog commit-a. Kod je mjerodavan ako se nešto kasnije promijeni.

Sadržaj

1 Uvod i namjena dokumenta

2 Arhitektura sistema

3 Model izvršavanja: niti, brave i razmjena podataka

4 Backend (C++): moduli i algoritmi

- 4.1 DroneLink: MSP veza sa kontrolerom leta
- 4.2 Senzorske klase
- 4.3 SerialPortScan: prepoznavanje portova
- 4.4 CRSF telemetrija: CrsfProtocol, CrsfStateMapper, CrsfLink
- 4.5 Komande sa računara: UplinkProtocol, CommandLink, radcmd.lua
- 4.6 VideoLink: snimanje, prikaz i OSD
- 4.7 DetectionLink i ObjectTracker
- 4.8 Bindings: granica C++ / Python

5 Frontend (Python): aplikacija, radne niti i paneli

- 5.1 DroneCockpitApp: pokretanje, petlja, gašenje
- 5.2 TelemetryWorker: izbor izvora i prevod podataka
- 5.3 VideoWorker i DetectionWorker
- 5.4 Prilagođavanje vezi: link_mode i dvoslojni paneli
- 5.5 Instrumenti i prozori
- 5.6 Upravljanje sa računara: CommandPanel, UplinkController, AltitudeHold

6 Tokovi podataka od kraja do kraja

7 Otpornost na otkaze i sigurnost

8 Testiranje

9 Dodatak: konstante za podešavanje i pojmovnik

1 Uvod i namjena dokumenta

R.A.D. je kontrolna stanica za izviđački dron na F450 ramu. Na jednom Windows računaru istovremeno prikazuje telemetriju kontrolera leta, analogni FPV video sa softverskim OSD-om, otkriva i prati objekte na videu (YOLO), procjenjuje njihovu geografsku poziciju i, od skora, šalje ograničene komande dronu preko radio-predajnika.

Dokument je namijenjen programeru koji preuzima ili pregleda kod. Odgovara na pitanja: koji modul radi šta, kojim metodama i algoritmima, u kojoj niti se šta izvršava, kako se podaci parsiraju i prosljeđuju dalje i šta se dešava kada nešto otkaže. Ne zamjenjuje komentare u kodu ni stranice u `docs/`, nego ih povezuje u jednu cjelinu.

Kako čitati

- **Poglavlja 2 i 3** daju cijelu sliku: procese, module i niti. Pročitati ih prije svega ostalog.
- **Poglavlje 4** opisuje C++ module redom kojim podaci ulaze u sistem: kontroler leta, radio, video, detekcija, pa sloj prema Python-u.
- **Poglavlje 5** opisuje Python stranu: aplikaciju, radne niti, panele i upravljanje sa računara.
- **Poglavlja 6 i 7** prate konkretne tokove (telemetrija, detekcija, komanda) i reakcije na otkaze.

Nazivi klasa, metoda i konstanti pišu se `ovako`, tačno kao u kodu. Python nazivi su u `snake_case` obliku, C++ u `camelCase`; pybind11 sloj ih preslikava (npr. `getLatestState()` → `get_latest_state()`).

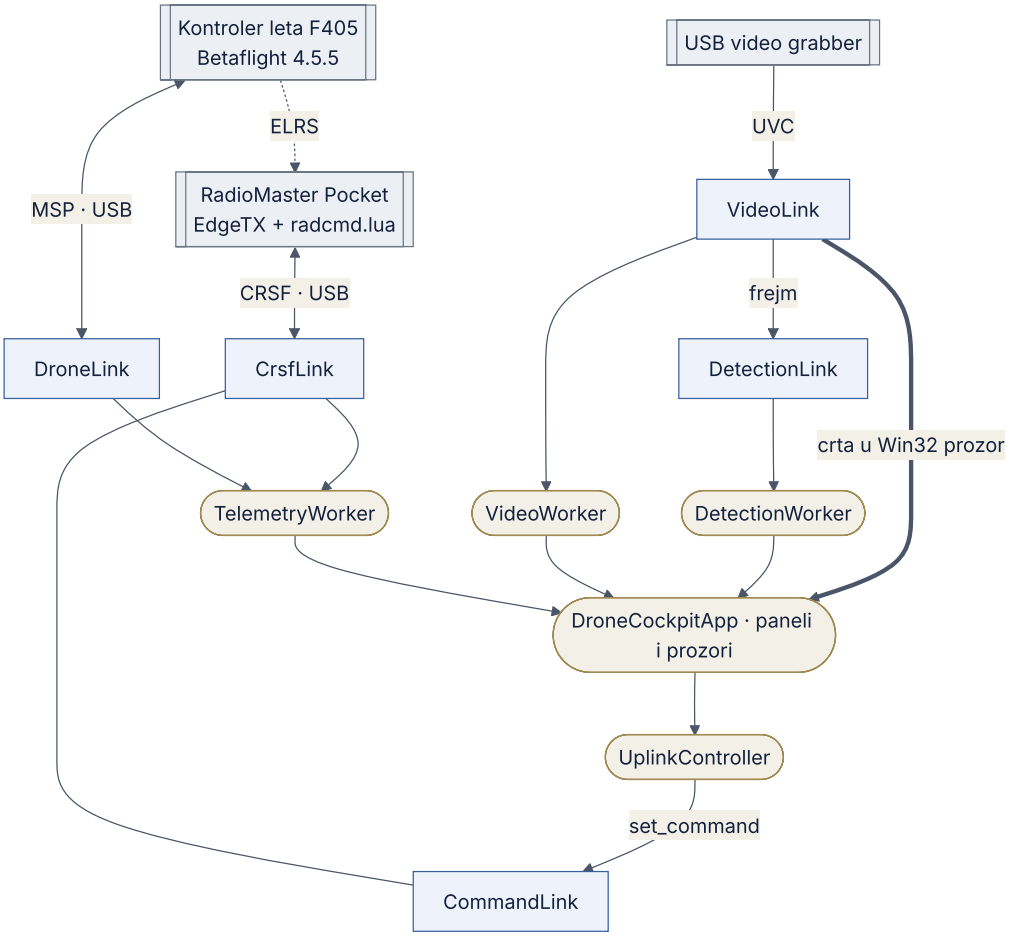
2 Arhitektura sistema

2.1 Jedan proces, dva jezika

Cijela kontrolna stanica radi kao **jedan Windows proces**. Podijeljena je po jasnom pravilu:

- **C++ jezgro** se kompajlira u Python ekstenziju `DroneBackend.pyd` (pybind11, Visual Studio, Release x64). Ovdje je sve što dira hardver, mora biti brzo ili radi sa pikselima: serijska komunikacija (MSP i CRSF), snimanje videa, crtanje preko Win32 GDI i YOLO inferencija preko OpenCV DNN-a (CUDA 13.3, cuDNN 9.24).
- **Python kokpit** (`DroneCockpitUI.py` , Tkinter, klasa `DroneCockpitApp`) drži prozor, pomične panele instrumenata i interakciju sa korisnikom. Nikad ne otvara serijski port niti poziva OpenCV direktno: svaki podatak sa hardvera dobija preko pybind11 klasa.

Namjerno pravilo: **pikseli ne prelaze u Python**. Video se crta direktno u nativni Win32 prozor iz C++ niti. Postoje samo dva uska izuzetka: JPEG pregled detekcija (`get_latest_annotated_frame_jpeg()` , samo dok je otvoren pregled) i `get_latest_frame()` za buduće snimanje.



Slika 1. Moduli, hardver i glavni tokovi podataka. Sivo: hardver; pravougaonici: C++; zaobljeno: Python.

2.2 Raspored foldera

```
Project R.A.D/
├── DroneBackend/                C++ → DroneBackend.pyd
│   ├── src/core/                DroneLink (MSP, DroneState), SerialPortScan
│   ├── src/sensors/             GPSneoM10, IMUSensor, MagQMC5883L, BaroBMP280
│   ├── src/radio/               CrsfProtocol, CrsfStateMapper, CrsfLink,
│   │                            UplinkProtocol, CommandLink
│   ├── src/video/               VideoLink
│   ├── src/detection/           DetectionLink, ObjectTracker
│   ├── src/bindings/            Bindings.cpp, UplinkBindings.h
│   └── tests/                   test_uplink.cpp, tracker_sim.cpp, detectionlink_run.cpp
├── DroneCockpitUI/              Python/Tk kokpit
│   ├── DroneCockpitUI.py        ulazna tačka, DroneCockpitApp
│   ├── widgets/                 paneli instrumenata, command_panel, OSD kontrole
│   ├── workers/                 telemetry/video/detection worker, uplink_controller
│   ├── core/                    link_mode, dual_layer, MapTiles, ui_perf
│   ├── control/                 altitude_hold
│   ├── config/                  rasporedi panela, OSD raspored (JSON)
│   ├── tools/                   alati za klupu i podešavanje
│   ├── tests/{unit,integration,system}
│   └── NNTraining/              podaci i obuka modela
├── Firmware/pocket/SCRIPTS/MIXES/radcmd.lua    EdgeTX most za komande
├── Firmware/betaflight/         CLI backup-i
└── docs/                        dokumentacija po modulima
```

2.3 Moduli i veličina

Modul	Datoteke	Linija	Odgovornost
`DroneLink`	core/DroneLink.h/.cpp	1557	MSP veza preko USB-a, 100 Hz prozivanje, `DroneState`
`SerialPortScan`	core/SerialPortScan.h/.cpp	229	popis COM portova sa USB imenima, MSP prepoznavanje kontrolera
senzori	sensors/*	1023	parsiranje GPS, IMU, magnetometra i barometra iz MSP poruka
`CrsfProtocol`, `CrsfStateMapper`	radio/Crsf*	815	CRSF okviri, CRC, dekoderi, preslikavanje u `DroneState`
`CrsfLink`	radio/CrsfLink.h/.cpp	525	port Pocket-a, nit čitača, ponovno spajanje, status veze
`UplinkProtocol`, `CommandLink`	radio/Uplink*, CommandLink*	523	komandni okviri, ograničenja, lease, stanje Lua mosta
`VideoLink`	video/VideoLink.h/.cpp	2852	snimanje, stanje video veze, crtanje u Win32 prozor, OSD
`DetectionLink`, `ObjectTracker`	detection/*	2929	YOLO, praćenje objekata, georeferenciranje, zapisi
bindings	bindings/*	1357	pybind11 modul `DroneBackend`
`DroneCockpitApp`	DroneCockpitUI.py	3680	prozor, paneli, petlja osvježavanja, spajanje, gašenje

Modul	Datoteke	Linija	Odgovornost
workers	workers/*	1109	pozadinske Python niti prema C++ objektima
widgets	widgets/*	9968	instrumenti, mapa detekcija, OSD postavke, panel komandi
core, control	core/*, control/*	625	pomoćna logika za vezu, slojeve, mape, mjerenje, držanje visine
`radcmd.lua`	Firmware/pocket/...	383	most na radio-predajniku: komande u mikser, telemetrija nazad

2.4 Dva izvora telemetrije

Kokpit prima podatke kontrolera leta na dva načina, a oba popunjavaju **istu strukturu** `DroneState`. Zbog toga instrumenti ne moraju znati odakle podatak dolazi.

Izvor	C++ klasa	Put	Sadržaj
`"USB"`	`DroneLink`	MSP prozivanje preko USB-C kabla	sve: sirovi IMU i magnetometar, motori, RC kanali, lista satelita, HDOP, razlozi blokade naoružavanja, opterećenje CPU-a
`"ELRS"`	`CrsfLink`	FC → prijemnik RP4TD → ELRS 2,4 GHz → Pocket → USB-VCP → računar	stav, baterija, GPS (bez HDOP-a i tipa fiksa), baro i vario, režim leta, statistika radio veze

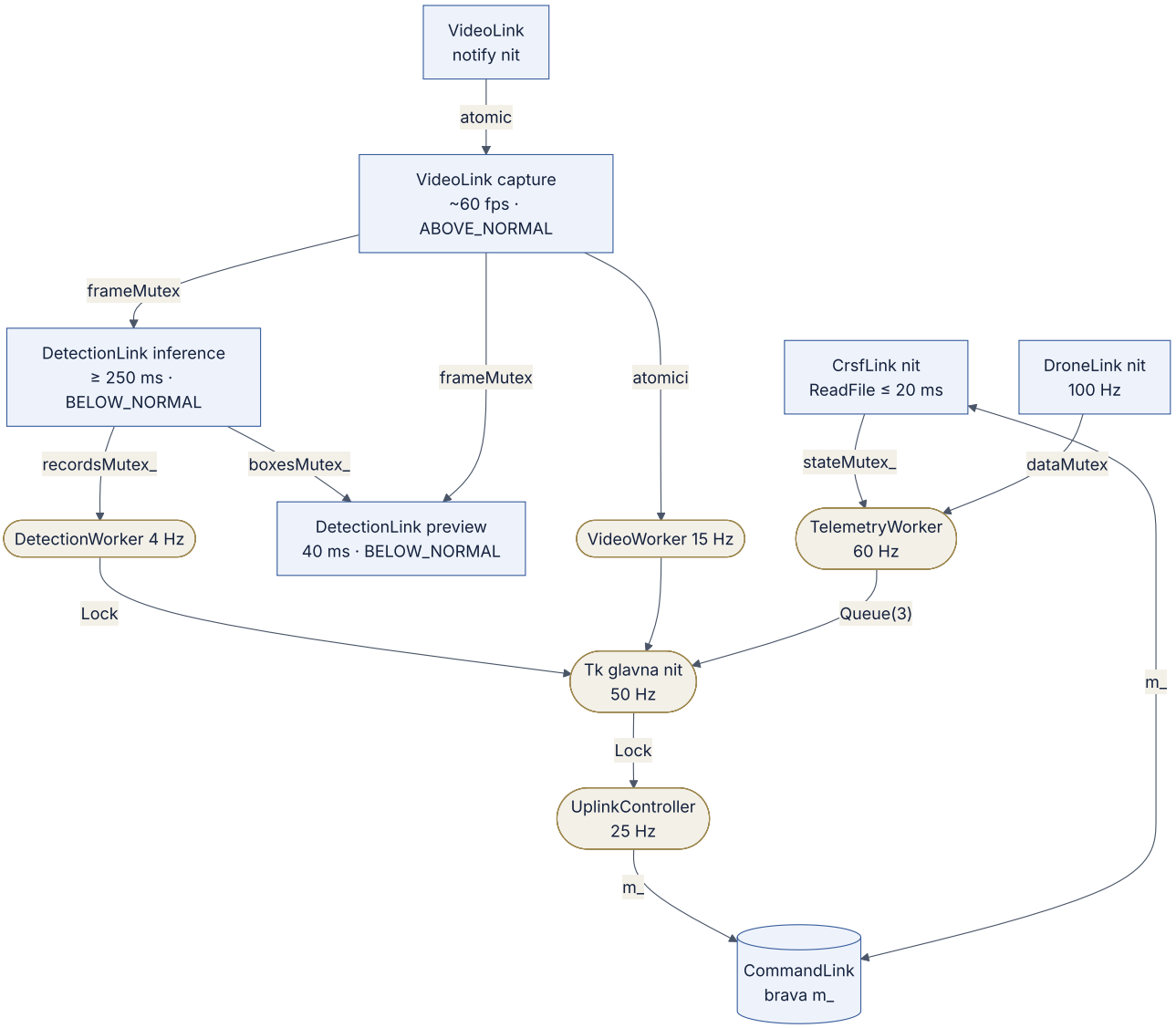
Python bira izvor u svakom ciklusu (odjeljak 5.2). Kada veza nestane, ništa se ne briše: zadnje vrijednosti, posebno zadnja GPS pozicija, ostaju na ekranu i na mapi detekcija. Za spasilačku misiju je mjesto gdje je veza pukla najvažniji podatak.

3 Model izvršavanja: niti, brave i razmjena podataka

Sistem radi u realnom vremenu: video, telemetrija, detekcija i komande moraju teći istovremeno i nijedan dio ne smije čekati drugi. To se postiže jednim dosljednim obrascem: **svaki hardverski modul ima svoju nit, a niti razmjenjuju samo zadnju vrijednost pod kratkom bravom**.

3.1 Sve niti u procesu

Nit	Vlasnik	Ritam	Prioritet	Posao
`communicationLoop()`	DroneLink (C++)	10 ms (100 Hz)	normalan	MSP upiti, parsiranje, objava `DroneState`
`workerLoop()`	CrsfLink (C++)	`ReadFile` ≤ 20 ms; status 10 Hz	normalan	traženje Pocket-a, CRSF parsiranje, slanje komandi
`captureLoop()`	VideoLink (C++)	ritam kamere (~60 fps)	ABOVE_NORMAL	snimanje, stanje veze, crtanje + OSD
`notifyThreadMain()`	VideoLink (C++)	na događaj	normalan	Windows poruke o priključenju uređaja
`previewLoop()`	DetectionLink (C++)	40 ms, samo dok se pregled proziva	BELOW_NORMAL	frejm + zadnji okviri detekcija za pregled
`inferenceLoop()`	DetectionLink (C++)	≥ 250 ms + odmor	BELOW_NORMAL	YOLO, praćenje, georeferenciranje, zapisi
Tk glavna nit	DroneCockpitApp	pumpa 20 ms (50 Hz) + događaji	–	crtanje panela, tastatura, miš
`TelemetryWorker._run`	Python	60 Hz	–	izbor izvora, `DroneState` → `ui_data`
`UplinkController._run`	Python	25 Hz	–	provjere, držanje visine, `set_command()`
`VideoWorker._loop`	Python	15 Hz	–	samo status videa (bez piksela)
`DetectionWorker._loop`	Python	4 Hz	–	novi zapisi detekcija i indeks frejmova
`UsbProbe` , `UsbDrop`	Python	jednokratno	–	traženje kontrolera, prekid USB veze



Slika 2. Mapa niti. Pravougaonici su C++ niti, zaobljeni oblici Python niti; uz svaku strelicu je brava koja štiti prenos.

3.2 Obrazac razmjene: zadnja vrijednost, ne red čekanja

Svaka predaja između niti ima isti oblik: proizvođač prepisuje **zadnju** vrijednost pod malom bravom (ili `std::atomic` za skalare), a potrošač je kopira. Nema blokirajućih redova i nema čekanja na drugu stranu.

Podatak	Piše	Čita	Zaštita
`DroneLink::currentState`	communicationLoop	bilo koja nit, `getLatestState()`	`dataMutex`, vraća se kopija
`CrsfLink::state_`, `mapper_`	workerLoop	bilo koja nit	`stateMutex_`; prije kopije se poziva `refresh()` pa su starosti tačne
`CommandLink` stanje	UplinkController (komanda), CrsfLink worker (okviri)	obje strane, UI	`m_`

Podatak	Piše	Čita	Zaštita
`VideoLink::latestFrame`	captureLoop	DetectionLink, Python	`frameMutex` samo za zaglavlje `cv::Mat`; kopija piksela poslije otpuštanja brave
`DetectionLink::latestBoxes_`	inferenceLoop	previewLoop	`boxesMutex_`
zapisi detekcija	inferenceLoop	DetectionWorker	`recordsMutex_`; snimci ekrana se pišu van brave
`ui_data`	TelemetryWorker	Tk nit	`queue.Queue(maxsize=3)`; pun red izbacuje najstariji
status veze	TelemetryWorker	Tk nit	zamjena cijelog `dict`-a (atomska dodjela reference)
intencije pokreta	Tk nit	UplinkController	`threading.Lock`

Jedini red je onaj između `TelemetryWorker`-a i Tk niti, i on je ograničen na 3 elementa. `get_frame()` isprazni cijeli red i vrati samo zadnji element, pa zauzeta Tk nit nikad ne zaostaje: na sljedećem čitanju odmah dobija najsvježije stanje.

3.3 Zašto nema uzajamnog blokiranja (deadlock)

- Brave se **nikad ne drže jedna unutar druge**. Na primjer, u `CrsfLink`-u se `stateMutex_` i `cfgMutex_` zaključavaju odvojeno, u kratkim blokovima.
- **Nijedna brava se ne drži dok traje I/O** (`ReadFile`, `WriteFile`, upis snimka na disk).
- C++ radne niti **ne traže Python GIL** za svoj posao. Izuzetak su povratni pozivi prema Python-u (odjeljak 3.5).
- Svi blokirajući pozivi iz Python-a u C++ (spajanje, traženje porta, kodiranje JPEG-a) **otpuštaju GIL** (`py::gil_scoped_release`), pa Tk nit ne staje dok traju.

3.4 Python niti i GIL

Python niti dijele jedan GIL. Dug posao u jednoj Python niti može zakasniti drugu, a C++ niti to ne pogađa. Zbog toga:

- Sav težak posao (video, detekcija, serijski portovi) je u C++ nitima bez GIL-a.
- `DroneCockpitUI.py` pri pokretanju postavlja `sys.setswitchinterval(0.002)`. Izmjereno sa dvije CPU-zauzete Python niti: najgori period kontrolera komandi pao je sa 130–240 ms (podrazumijevanih 5 ms) na 65–85 ms (cilj 40 ms, lease 300 ms).
- Radio tok ne zavisi od Python tajminga: C++ nit ponavlja zadnju komandu oko 33 puta u sekundi, a Python kontroler je samo osvježava.

3.5 Povratni pozivi iz C++ u Python (trampolini)

`VideoLink` (za OSD) i `DetectionLink` (za georeferenciranje) trebaju telemetriju, ali ne znaju ništa o `DroneLink`-u ni `CrsfLink`-u. Dobijaju je preko `setTelemetryProvider(std::function<TelemetrySnapshot()>)`. Python im predaje dvije različite funkcije:

	`_get_detection_telemetry()`	`_get_osd_telemetry()`
Predaje se	`DetectionLink`	`VideoLink`
Poziva se iz	inferenceLoop, jednom po prolazu	captureLoop, jednom po nacrtanom frejmu
`valid` znači	GPS pozicija upotrebljiva; inače nema georeferenciranja	veza sa kontrolerom postoji (OSD-u ne treba GPS fiks)
`altitude_m`	barometar, visina iznad tačke naoružavanja	barometar (isto kao `BaroWidget`)
ugao kamere	ručno podešen ugao iz trake alata (gimbal još nije ožičen)	ne koristi se

pybind11 omotač (`functional.h`) **automatski uzima GIL** pri svakom pozivu iz C++ niti. Obje funkcije su kratke: uzimaju kopije stanja iz `TelemetryWorker.get_active_state()` i pune strukturu.

> **Poznata sprega:** `captureLoop` (ABOVE_NORMAL) poziva OSD trampolin za svaki frejm i pritom čeka GIL. Ako neka Python nit drži GIL, crtanje tog frejma može zakasniti do jednog intervala preključivanja (sada 2 ms). Uticaj je mali, ali ako bi OSD ikad postao usko grlo, rješenje je da `TelemetryWorker` sam gura gotov `TelemetrySnapshot` u C++ (atomska zamjena), pa capture nit uopšte ne ulazi u Python.

4 Backend (C++): moduli i algoritmi

4.1 DroneLink: MSP veza sa kontrolerom leta

`DroneLink` drži USB serijsku vezu sa Betaflight kontrolerom (57600 Bd, `MSP_BAUD`), proziva sve poruke koje kokpitu trebaju i objavljuje jedan zaštićen snimak `DroneState`.

Životni ciklus

```
connect(port) → openSerialPort() → std::thread(communicationLoop)
disconnect()  → keepRunning = false → join() → closeSerialPort()
```

Okvir i čitanje odgovora: `sendMSP(id)`

Koristi se MSP v1 bez korisnog tereta u upitu: `$ M < 0x00 id id` (dužina 0, komanda, kontrolna suma = XOR dužine i komande). Čitanje odgovora je napravljeno tako da preživi zaostale bajtove od ranijih upita:

1. Rok za cijeli odgovor je **80 ms**. Bajtovi se čitaju jedan po jedan, sa pauzom od 150 µs kad bafer prazan.
2. **Sinhronizacija**: traži se zaglavlje `$ M >`, najviše 64 pokušaja. Time se odbacuju ostaci prethodnog odgovora bez `PurgeComm`-a, koji bi obrisao i bajtove trenutnog odgovora u prenosu.
3. Čitaju se dužina i komanda. Ako komanda nije ona tražena, ovo je zaostali okvir: tada se radi `PurgeComm(PURGE_RXCLEAR)` i vraća prazan rezultat.
4. Čitaju se teret i kontrolna suma. Rezultat je cijeli okvir koji parser dalje provjerava.

Za testove postoji `setFailInjection(true)`: `sendMSP` tada odmah vraća prazno i deterministički simulira mrtvu vezu.

Petlja prozivanja: `communicationLoop()`

Svaki ciklus (10 ms) radi sljedeće, redom:

1. Kopira `currentState` u lokalni `pending` (pod `dataMutex`). Svi parseri pišu samo u `pending`.
2. Obrađuje zahtjeve za kalibraciju. UI nit samo podiže atomsku zastavicu (`magCalRequested`, `accCalRequested`); petlja je preuzme sa `exchange(false)`, pošalje `MSP_MAG_CAL` ili `MSP_ACC_CAL` i odbrojava (30 s, odnosno 5 s) u `DroneState`.
3. Šalje redovne upite i parsira odgovore: `STATUS`, `RAW_IMU` (uz mjerenje RTT-a), `ATTITUDE`, `ANALOG`, `DEBUG`, `ALTITUDE`, `RAW_GPS`, `COMP_GPS`, `NAV_STATUS`, `STATUS_EX`, `MOTOR`, `RC`.
4. Svakih 100 ciklusa (~1 s) proziva listu satelita (`MSP_GPS_SV_INFO`), a svakih 50 ciklusa (~0,5 s) detaljno stanje baterije (`MSP_BATTERY_STATE`).
5. Prati zdravlje veze: ako je IMU odgovor uspio, `linkHealthy = true` i brojač paketa raste; poslije 5 uzastopnih neuspjeha (`FAIL_THRESHOLD`) veza se proglašava nezdravom.
6. `commitState(pending)` atomski objavljuje novo stanje pod `dataMutex`, pa spava ostatak od 10 ms.

> Oko 13 upita po 1–2 ms čini budžet od 10 ms tijesnim na sporijem hardveru. Ako petlja ne stiže, `setPollIntervalMs(20)` spušta prozivanje na 50 Hz.

Parseri

MSP	Parser	Popunjava
101 STATUS	`parseStatus`	naoružan, zastavice i ime režima, senzori, I2C greške, CPU, PID profil
102 RAW_IMU	`parseIMU`	sirovi akcelerometar i žiroskop (ADC jedinice)
104 MOTOR / 105 RC	`parseMotors` , `parseRCChannels`	izlazi motora, do 18 RC kanala
106 / 107 / 121	preko `GPSNeoM10`	fiks, sateliti, pozicija, brzina, kurs, HDOP, udaljenost i smjer do kuće, status navigacije
108 ATTITUDE	`parseAttitude`	roll i pitch (decistepeni), yaw (stepeni)
109 ALTITUDE	preko `BaroBMP280`	procijenjena visina (cm) i vertikalna brzina (cm/s)
110 ANALOG	`parseAnalog`	napon, struja, potrošnja mAh, RSSI
150 STATUS_EX	`parseStatusEx`	maska i tekst razloga blokade naoružavanja
164 GPS_SV_INFO	`pollSatellitesMSP`	lista satelita (`svSource = "MSP"`)
242 BATTERY_STATE	`parseBatteryState`	broj ćelija, kapacitet, procenat, stanje baterije
254 DEBUG	`parseDebug`	sirovi magnetometar i kurs (zahtijeva `debug_mode = MAG_CALIB`)

`decodeFlightMode()` i `decodeArmingDisable()` pretvaraju bitske maske u tekst za UI (npr. `"ANGLE [DISARMED]"` , `"GPS RESCUE"`).

Konfiguracija GPS-a preko passthrough-a

`applyGPSConfig()` je jedini put kojim kokpit razgovara direktno sa NEO-M10 modulom. Pošalje `MSP_SET_PASSTHROUGH` sa indeksom GPS UART-a; kontroler potvrdi i počne prosljeđivati sirove bajtove. Zatim se šalju UBX okviri (`CFG-GNSS` , `CFG-RATE` , `CFG-PRT` , `CFG-NAV5` , `CFG-CFG`) i čitaju UBX potvrde. Betaflight 4.x nikad sam ne izlazi iz passthrough-a, pa se port na kraju zatvori i ponovo otvori (USB čip se resetuje za ~150–250 ms).

4.2 Senzorske klase

Ove klase nemaju nit ni vezu. Pozivaju se sinhrono iz petlje `DroneLink` -a i služe da se parsiranje može testirati odvojeno.

- **`GPSNeoM10`** (samo statičke metode): MSP parseri za GPS i UBX graditelji okvira za passthrough. Važan detalj formata `MSP_GPS_SV_INFO` u Betaflight-u 4.5: zapis od 4 bajta je `gnssId, svid, flags, cno` . Treći bajt sadrži `qualityInd` (bitovi 2:0) i `svUsed` (bit 3), a **ne** upakovan `gnssId` . Elevacija i azimut u ovom režimu ne postoje, pa ih UI skriva.
- **`IMUSensor`** : pretvara sirove jedinice u fizičke. Akcelerometar 1/2048 g po jedinici (MPU-6500 na ±16 g), žiroskop 1/16,4 °/s (±2000 °/s). Ranija vrijednost 1/8192 je davala 4× manje očitavanje (defekt DEF-002, test IT-IMU-002).
- **`MagQMC5883L`** : kurs `atan2(y, x)` normalizovan na 0–360°, bez kompenzacije nagiba, i MSP XOR kontrolna suma. Betaflight 4.x nema `MSP_RAW_MAG` : ID 130 je preuzeo `MSP_BATTERY_STATE` , pa se

sirovi magnetometar čita preko `MSP_DEBUG`.

- **`BaroBMP280`**: dekodira `MSP_ALTITUDE` (int32 visina u cm + int16 vario) i preračunava u metre, stope i QNH-korigovanu visinu.

4.3 SerialPortScan: prepoznavanje portova

Kada su priključeni i kontroler i Pocket, računar vidi **dva STM32 serijska uređaja**, često sa istim VID:PID (`0483:5740`). Modul ih razlikuje na dva načina:

- `EnumerateSerialPorts()` preko SetupAPI-ja čita za svaki prisutni port i **USB ime proizvoda** (`DEVPKEY_Device_BusReportedDeviceDesc`, npr. „Betaflight STM32F405", „EdgeTX ...").
- `AutoDetectFlightController()` prihvata port **samo ako odgovori na MSP**. Prvo proba portove koji se zovu Betaflight, preskače isključene, radio i Bluetooth portove, otvara svaki na 57600 8N1 sa ugašenim DTR/RTS, šalje `MSP_API_VERSION` i čeka `$M>` sa komandom 1 najviše 300 ms. Zato nikad ne može izabrati Pocket.

Detekcija blokira do 300 ms po portu, pa je kokpit pokreće u posebnoj niti `UsbProbe`, a binding otpušta GIL.

4.4 CRSF telemetrija: CrsfProtocol, CrsfStateMapper, CrsfLink

Put podataka je: Betaflight → prijemnik RP4TD → ELRS 2,4 GHz → Pocket → USB-VCP → `CrsfLink` → `DroneState` → instrumenti. `CrsfProtocol` i `CrsfStateMapper` su čista logika, bez Win32 koda, niti i I/O-a, pa se testiraju bez radija. Sav Windows kod radio dijela je u `CrsfLink`-u.

Okvir

```
[adresa/sync][len][tip][teret ...][crc8]
  len  = 1 (tip) + teret + 1 (crc), 2 ≤ len ≤ 62, najviše 64 bajta ukupno
  crc8 = CRC-8/DVB-S2 (polinom 0xD5, početak 0x00) nad tipom i teretom
  višebajtna polja su big-endian (MSP je little-endian)
```

Prihvaćeni početni bajtovi: `0xC8` (kontroler, opšti sync), `0xEA`, `0xEC`, `0xEE`.

Algoritam parsiranja toka: `StreamParser::feed()`

Prima komade proizvoljne veličine, tačno kako ih vrati `ReadFile`, i dodaje svaki ispravan okvir u izlaznu listu:

- **okvir podijeljen između čitanja**: nepotpuni rep ostaje u baferu do sljedećeg poziva;
- **smeće između okvira**: bajtovi se preskaču dok sync bajt ne prati smislena dužina;
- **pogrešan CRC**: pomjera se samo **jedan bajt** naprijed i ponovo traži sync, pa se ne gubi pravi okvir koji počinje unutar oštećenog;
- **čisti šum**: bafer se briše kad pređe 4 × 64 bajta.

Dekoderi

Tip	ID	Sadržaj i jedinice
GPS	0x02	lat/lon (×1e7), brzina km/h (×10), kurs (×100), visina (+1000 m pomak), sateliti
VARIO	0x07	vertikalna brzina cm/s

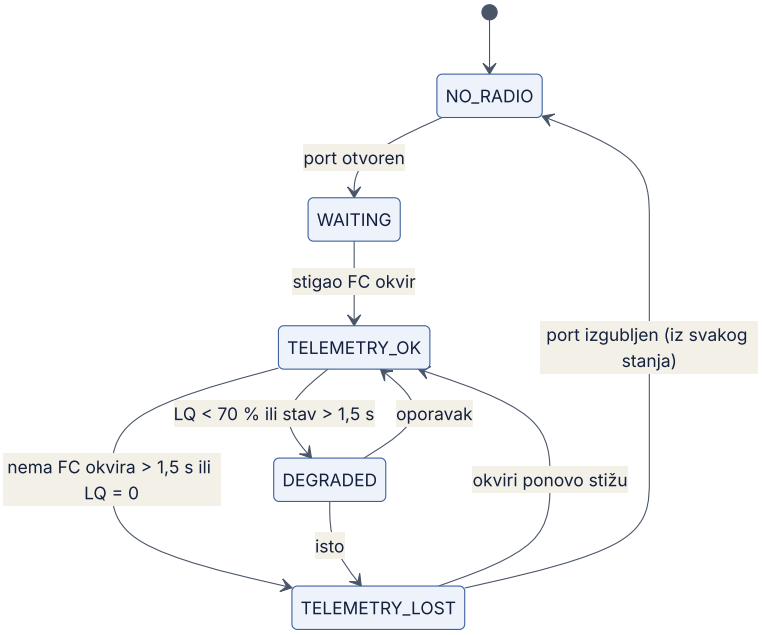
Tip	ID	Sadržaj i jedinice
BATTERY	0x08	V i A (0,1), mAh (uint24), procenat
BARO_ALTITUDE	0x09	bit 15: grubi režim u metrima; inače decimetri sa pomakom +10000
LINK_STATISTICS	0x14	RSSI obje antene (ELRS šalje bajt sa predznakom), LQ, SNR, antena, RF režim, snaga predajnika, downlink vrijednosti
ATTITUDE	0x1E	pitch, roll, yaw (int16 radijani × 10000), yaw normalizovan na [0, 360)
FLIGHT_MODE	0x21	ASCII tekst, npr. `STAB`, `ACRO*` (zvjezdica = nenaoružan)

Preslikavanje u `DroneState`: `CrsfStateMapper`

`apply(frame, state, now)` se poziva za svaki okvir, a `refresh(...)` oko 10 puta u sekundi i prije svake kopije za Python. Mapper pretvara sve u **iste jedinice koje koristi MSP**, pa instrumenti rade bez izmjena (roll i pitch u decistepenima, brzina u cm/s, kurs u decistepenima). Pored toga sam računa ono što CRSF ne nosi:

- **Režim leta**: `STAB` → ANGLE, `HOR` → HORIZON, `RTH` → GPS RESCUE, `!FS!` → FAILSAFE, `!ERR` → blokada naoružavanja; zvjezdica na kraju znači nenaoružan.
- **Tačka kuće**: na svakoj ivici naoružavanja stara kuća se briše, a prvi upotrebljiv GPS fiks dok je dron naoružan postaje nova kuća. Udaljenost se računa haversinom, a smjer kao početni azimut velikog kruga.
- **Fiks**: CRSF nema tip fiksa ni HDOP. Fiks je 2D ako je pozicija različita od nule uz ≥ 4 satelita, a „upotrebljiv" uz ≥ 5 (`minSatsForFix`). HDOP je uvijek 9999.
- **Baterija**: ako broj ćelija nije zadat, procijeni se jednom iz prvog očitavanja kao `ceil(V / 4,35)`. Pragovi po ćeliji su 3,5 V (upozorenje) i 3,3 V (kritično). Kokpit zadaje 4 ćelije (`RADIO_BATTERY_CELLS`).
- **Rezervna visina**: ako baro okvir nije stigao 2 s, visina = (GPS visina − visina kuće) × 100, a `altitudeSource = "GPS"`.
- **Starosti i učestanosti** po grupi podataka (−1 = nikad), u kliznom prozoru od oko 2 s.

Status veze



Slika 3. Stanja radio veze (`evaluateStatus()`). „FC okvir" znači stav, baterija, režim leta ili GPS; `LINK_STATISTICS` može generisati i sam predajnik, pa ne dokazuje da je dron živ.

Radna nit: `CrsfLink::workerLoop()`

```
nije spojen ─► traži Pocket (auto) ili ponovo otvori zadati port, svakih 1000 ms
spojen      ─► ReadFile (vraća se čim stigne bajt, najkasnije za 20 ms)
            ─► StreamParser.feed ─► mapper.apply(okvir)       (pod stateMutex_)
            ─► serviceUplink(okviri)                          (komande, odjeljak 4.5)
            ─► svakih 1 s:   SerialPortExists(port)?          (provjera iskopčavanja)
            ─► svakih 100 ms: mapper.refresh()                (starosti, učestanosti, status)
```

Izbor porta (`scanForRadio()`): port sa isključenja, Bluetooth i portovi koji se zovu Betaflight ili INAV se preskaču. Port čije USB ime sadrži `edgetx` , `opentx` , `radiomaster` ili `pocket` prihvata se i ako ćuti (dron još ugašen). Bilo koji drugi port se prihvata samo ako za 500 ms stignu bar 2 ispravna CRSF okvira. Svako skeniranje ostavlja čitljiv izvještaj (`getLastScanReport()`).

Iskopčavanje: port se odbacuje ako `ClearCommError` ili `ReadFile` ne uspije, ili ako COM uređaj nestane iz `QueryDosDevice` (Windows ponekad drži mrtvu ručku „ispravnom"). Port prihvaćen samo zbog CRSF okvira pušta se ako ućuti 5 s, da veza ne bi držala port koji je možda kontroler.

4.5 Komande sa računara: UplinkProtocol, CommandLink, radcmd.lua

Pocket je prebačen u USB-VCP režim „LUA". Skripta `radcmd.lua` tada drži USB vezu u oba smjera: prima komande od računara i šalje telemetriju nazad, ponovo kodiranu u **iste CRSF okvire** koje je ranije slao „Telem Mirror". Zato `CrsfLink` parsira telemetriju bez izmjena.

Jedan vlasnik porta, dva smjera

`CommandLink` **nema port**. Drži „šta računar sada želi" i od toga pravi okvire. Port drži `CrsfLink` , čija radna nit u svakom krugu poziva `serviceUplink()` :

```
for (okvir : primljeni_okviri) up->onFrame(okvir, now);   // RAD_STATUS, RC_CHANNELS
if (up->frameDue(now)) {                                  // ~svakih 30 ms
    okvir = up->nextFrame(now);                           // ograničenja + slew
    WriteFile(port, okvir);  up->onWriteResult(ok);
}
```

Windows serijska ručka je sinhrona, pa bi čitanje i pisanje iz dvije niti ionako bilo serijalizovano. Jedna nit po portu je zato i najjednostavnije i ispravno rješenje. Komandni okvir čeka najviše jedan `ReadFile` timeout (20 ms).

Format komande (računar → Pocket)

Standardni CRSF `RC_CHANNELS_PACKED` (0x16, 16 kanala po 11 bita):

Kanal	Značenje
1 / 2 / 4	pomak roll / pitch / yaw (−100…100 %)
3	ciljni gas, apsolutno u µs
5	nagib gimbala (−100…100 %)
6 / 7	dozvola palica / dozvola gasa
8	brojač „živ", mijenja se u svakom okviru

Nazad stiže `RAD_STATUS` (0x7E, 22 bajta, samo preko USB-a): verzija, zastavice (svježe, prekidač, palice aktivne, gas aktivan, telemetrija živa), vraćeni brojač za mjerenje RTT-a, broj primljenih okvira, CRC greške, starost komande, pilotov gas, primijenjeni pomak gasa, period skripte i primijenjeni izlazi. Stiže i `RC_CHANNELS` sa kanalima koje Pocket stvarno šalje.

Zaštite u `CommandLink` -u

1. **Ništa se ne piše** dok `RAD_STATUS` ne dokaže da je na portu Lua most (nikad u Telem Mirror, CLI ili kontroler). Bez `RAD_STATUS` -a 1,5 s slanje prestaje.
2. **Glavna dozvola** (`setEnabled`): isključeno znači odmah ugašene palice i gas.
3. **Lease**: komanda važi 300 ms od zadnjeg `setCommand()` . Zaglavljena Python nit ne može ostaviti staru komandu aktivnom.
4. **Ograničenja**: roll i pitch ≤ 25 %, yaw ≤ 30 %, gas 1150–1750 µs. **Slew**: palice 150 %/s, gas 250 µs/s. Na uključenju gasa nema slew-a, da preuzimanje bude bez skoka.

Skripta na radiju: `radcmd.lua`

Mikser skripta EdgeTX-a; vrijeme mjeri u jedinicama od 10 ms (`getTime()`). Odluku donosi radio, ne računar:

- Svi izlazi su 0 osim ako je pilotov prekidač **SD** uključen i ako je **novi** okvir stigao u zadnjih 500 ms (`STALE_TICKS = 50`).
- Roll, pitch i yaw su **pomaci** koji se **dodaju** pilotovim palicama (mikser ADD), ograničeni na ±300 jedinica (~30 % palice).
- Gas je **razlika** u odnosu na pilotovu palicu, ograničena na ±200 jedinica (~±100 µs), i samo dok je palica iznad ~1200 µs. Gašenjem prekidača gas se vraća na palicu uz skok od najviše tog opsega.
- Naoružavanje, režimi leta i failsafe se ne diraju.

- Svaki krug šalje do 2 telemetrijska okvira (`TLM_PER_RUN`), da skripta ostane u CPU budžetu EdgeTX-a.

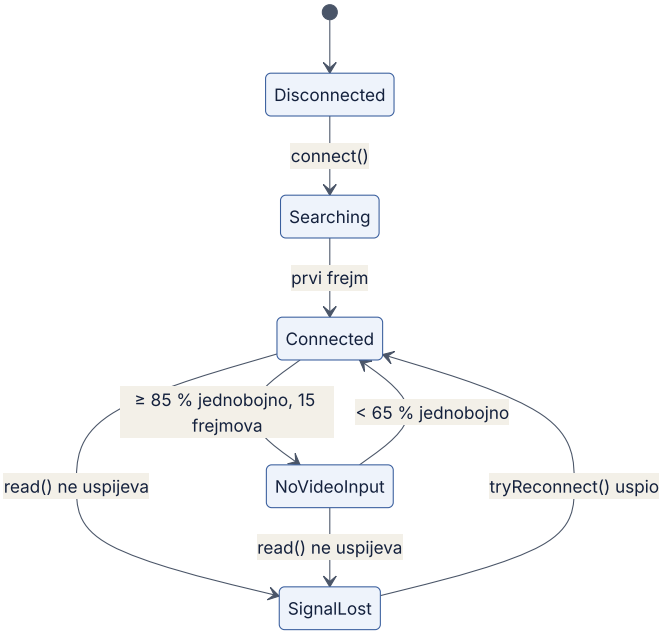
4.6 VideoLink: snimanje, prikaz i OSD

`VideoLink` snima frejmove sa USB grabera analognog videa, vodi pošten status veze za pilota i crta frejmove i softverski OSD **direktno u nativni Win32 prozor**, bez Python-a i Tk-a.

Spajanje

- `enumerateDevices()` navodi uređaje (Media Foundation se pokreće jednom po procesu).
- `connectAuto()` redom proba uređaje koji ne liče na ugrađenu web-kameru, dok jedan ne isporuči frejm.
- Otvaranje prvo proba `CAP_DSHOW` , pa `CAP_MSMF` , sa parametrima u samom `open()` da se izbjegne kašnjenje naknadnog podešavanja.
- Posebna nit sa skrivenim prozorom prima `WM_DEVICECHANGE` . Kad Windows javi priključen uređaj, `captureLoop` odmah pokušava ponovo spajanje umjesto da čeka svoj interval. `tryReconnect()` svaki put ponovo traži indeks po imenu uređaja, jer se indeks mijenja poslije ponovnog priključenja.

Stanje video veze



Slika 4. Stanja video veze (`getLinkState()`).

`NoVideoInput` se otkriva iz **sadržaja frejma**, ne iz greške čitanja. `idleColorFraction()` uzorkuje svaki 6. piksel u oba smjera i broji udio piksela unutar tolerancije 14 od srednje boje frejma. Za ulazak treba ≥ 0,85 u 15 uzastopnih frejmova (~250 ms), za izlazak < 0,65. Histereza sprečava treperenje kad se na praznoj slici mijenjaju samo cifre OSD-a. Nezavisno od toga, `getMsSinceLastFrame()` daje drugu provjeru zastarjelosti.

Petlja snimanja: `captureLoop()`

- Prioritet `THREAD_PRIORITY_ABOVE_NORMAL` .

- U stanju `SignalLost` pokušava spajanje na interval, a poslije obavještenja o priključenju neko vrijeme u ubrzanom ritmu.
- Svaki frejm se dekodira u **novi** `cv::Mat`. Zato `getLatestFrame()` pod bravom uzima samo zaglavlje, a kopiju piksela radi poslije, pa detekcija nikad ne koči snimanje.
- Crtanje: frejm i upozorenja se prvo slože u **pozadinski bafer** (`StretchDIBits`), pa se jednim `BitBlt`-om prenesu na prozor. Tako se izbjegava treperenje između pojedinačnih GDI poziva.

Softverski OSD

Zamjenjuje OSD kontrolera leta, koji je bio utisnut u analogni signal prije grabera i davao lažne detekcije. Softverski OSD se crta u isti pozadinski bafer, ali **poslije** spremanja frejma u `latestFrame_`, pa nikad ne ulazi u ulaz detekcije. Elementi (`altitude`, `horizon`, `compass`, tajmer, ...) imaju 9 sidara ili slobodnu poziciju; elementi na istom sidru se slažu bez preklapanja. Tajmer leta kreće pri naoružavanju, pauzira se pri razoružavanju i nastavlja dalje.

Usklađivanje sa Tk-om

Prozor za video je pravi Win32 child prozor van Tk stabla, pa ga Tk-ov `lift()` ne pomjera. Zato postoje `raiseWindow()`, `lowerWindow()`, `showWindow()` i `hideWindow()`, koje Python poziva kad se mijenja redoslijed ili vidljivost FPV panela. `WM_PAINT` boji otkrivenu površinu u crno kad video ne teče; `paintMutex_` serijalizuje crtanje između UI i capture niti.

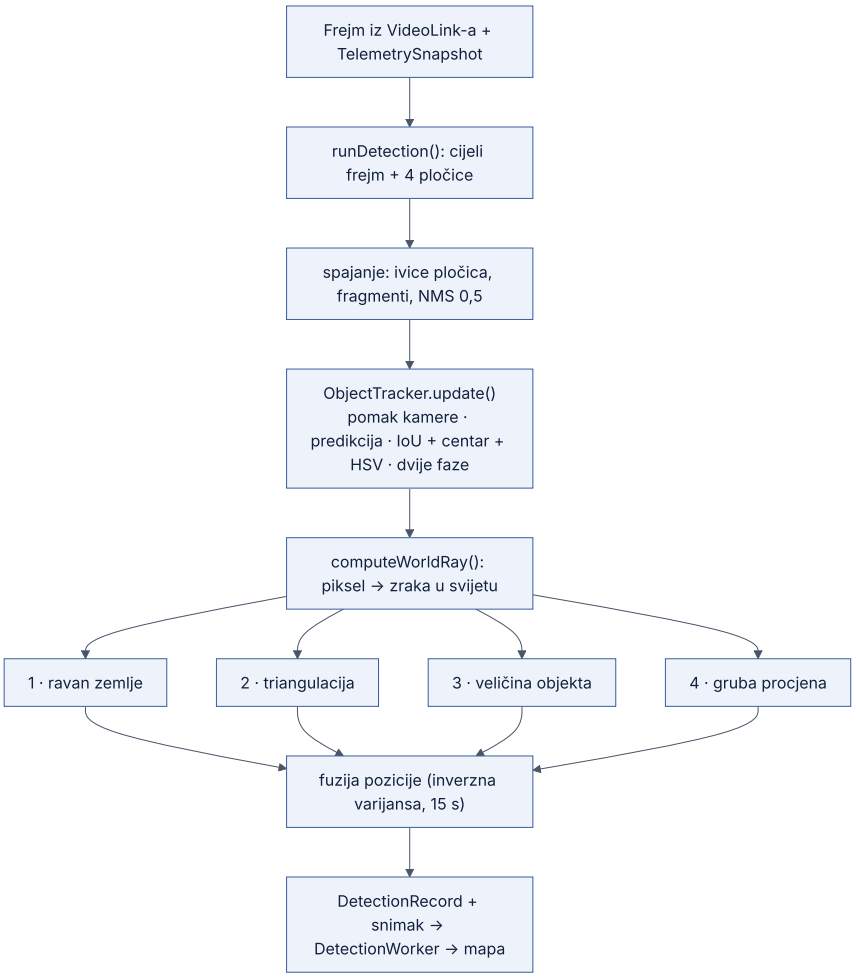
4.7 DetectionLink i ObjectTracker

Pokreće YOLO model nad videom bez ikakvog usporavanja pilotovog prikaza, prati objekte da jedan fizički objekat bude jedan trag, procjenjuje gdje se objekat nalazi i spaja procjene u jednu poziciju sa poluprečnikom nesigurnosti.

Dvije niti

- **`previewLoop()`** (40 ms, BELOW_NORMAL) uzima frejm i zadnje okvire detekcija i priprema pregled. **Nikad ne pokreće inferenciju**, pa je ne može zakočiti spor prolaz modela. Radi samo dok je pregled prozivan u zadnje 2 s.
- **`inferenceLoop()`** (BELOW_NORMAL) radi na `detectionIntervalMs_` (250 ms) ili sporije. Poslije svakog prolaza odmara najmanje prolaz × (1/d − 1), gdje je `d` najveći udio zauzetosti (0,66), dakle bar pola trajanja prolaza.

Jedan prolaz inferencije, korak po korak



Slika 5. Tok jednog prolaza detekcije.

1. Čeka da prođe interval i odmor; uzima frejm (`frameSource_` , obično `VideoLink::getLatestFrame()`) i odmah zatim `TelemetrySnapshot` , da budu što bliže u vremenu.
2. **`runDetection()`** : `runInference()` nad cijelim frejmom, a uz uključeno popločavanje i nad 4 ugaone pločice (60 % frejma, 20 % preklapanja). Spajanje: okvir koji završava do 2 % od unutrašnje ivice pločice se odbacuje; okvir pločice koji je ≥ 70 % unutar većeg okvira iste klase je fragment; na kraju NMS po klasi (IoU 0,5).
3. **`runInference()`** : letterbox na kvadrat `inputSize` (kokpit 960), `blobFromImage(1/255)` , `forward()` . Prepoznaju se dva ONNX izlaza: end-to-end `[1, N, 6]` (NMS u modelu) i sirovi `[1, 4+klase, sidra]` (argmax klase + NMS 0,7, najviše 300).
4. Kandidati ispod niskog praga (0,15) se odbacuju; ostali idu u **`ObjectTracker.update()`** .
5. Za uklonjene tragove sa poznatom pozicijom pravi se arhivski zapis za kasnije geografsko ponovno prepoznavanje.
6. Za svaki **potvrđen** trag: `computeGeoCandidate()` , dodavanje opažanja smjera u istoriju (najviše 12), ponovno prepoznavanje ili novi `trackId` , fuzija pozicije.
7. Zapis se piše za novi objekat, kad se pomjeri ≥ 3 m, svakih 10 s ili kad se georeferenciranje pojavi ili nestane (`shouldRecordAgain()`). Snimak ekrana se upisuje van brave.
8. Objavljuju se okviri za pregled (`boxesMutex_`) i stavka indeksa frejmova (koji su potvrđeni objekti bili zajedno u kadru).

Backend inferencije

Uz `setUseCuda(true)` pokušava se `DNN_TARGET_CUDA_FP16`; `start()` provjerava ga probnim prolazom (bez izuzetka, bez NaN/Inf u izlazu). Ako ne uspije, prelazi se na FP32 CUDA, pa na CPU. `isUsingCuda()` i `isUsingCudaFp16()` kažu šta je stvarno pokrenuto.

Algoritam praćenja: `trk::ObjectTracker`

1. **Pomak kamere**: `cv::phaseCorrelate` nad sivom kopijom širine 320 px trenutnog i prethodnog frejma, sa zamaskiranim okvirima detekcija, daje globalni pomak slike; svi tragovi se pomjere za njega.
2. **Predikcija**: svaki trag ima brzinu (alfa-beta filter po stvarnom vremenu) i predviđa se na sadašnji trenutak.
3. **Ocjena povezivanja** = IoU + udaljenost centara u veličinama objekta + sličnost HSV histograma (Bhattacharyya), unutar kapije koja raste što je trag duže neviđen. Pohlepno dodjeljivanje od najboljeg para.
4. **Grupe klasa**: klase vozila se mogu međusobno spojiti uz malu kaznu, osoba samo sa osobom. Klasa traga je glasanje po pouzdanosti kroz istoriju.
5. **Dvije faze** (ideja ByteTrack): prvo se povezuju detekcije iznad praga, zatim slabije (≥ 0,15) smiju samo produžiti već potvrđen trag.
6. **Životni ciklus**: probni → potvrđen poslije 2 pogotka (odmah uz pouzdanost ≥ 0,70) → izgubljen (i dalje se predviđa 5 s, najmanje 6 prolaza) → uklonjen. Samo potvrđeni tragovi postaju zapisi.

Simulator `tests/tracker_sim.cpp` mjeri broj ID-jeva po stvarnom objektu: za auto koji prelazi kadar pri 4 Hz stari algoritam (samo IoU) daje 20,9, novi 1,00.

Georeferenciranje

`computeWorldRay()` pretvara piksel u zraku u svijetu: pomak od centra i horizontalni FOV (kokpit 100°) daju zraku u kameri, zatim se primjenjuju pan i tilt kamere, pa roll, pitch i kurs drona. Konvencije: kurs 0 = sjever u smjeru kazaljke, roll pozitivan = desno krilo dolje, pitch pozitivan = nos gore, tilt 0 = naprijed, +90° = pravo dolje. Na istoj zraci zatim se redom probaju metode:

#	Metoda	Princip	Poluprečnik (~2σ)
1	ravan zemlje (`rangeByGroundPlane`)	presjek zrake sa ravni zemlje na visini drona; ne radi ispod 2 m, preko 400 m ni kad je zraka preplitka (komponenta naniže ≤ 0,12)	iz grešaka visine (3 m + 5 %), nagiba (4°) i kursa (5°)
2	triangulacija (`rangeByTriangulation`)	najmanji kvadrati nad smjerovima istog traga iz različitih pozicija; treba ≥ 5 m pomaka i ≥ 5° razlike smjerova	max(10 m, 20 % udaljenosti)
3	veličina objekta (`rangeByObjectSize`)	pinhole: udaljenost = stvarna širina × žižna daljina / širina okvira (osoba 0,5 m, auto 1,8 m …)	max(10 m, 35 %)
4	gruba procjena (`rangeCoarse`)	smjer kamere + udaljenost ravni zemlje ograničena na 150 m, ili 60 m kad ni ona nije poznata	≥ 50 m, tipično 60–150 m

Fuzija: svaki objekat drži srednju vrijednost svih georeferenciranih opažanja ponderisanu inverznom varijansom, sa poluživotom 15 s (da prati objekat koji se kreće). Poluprečnik se smanjuje sa brojem opažanja, ali nikad ispod pola najboljeg pojedinačnog, jer su greške montaže i kursa zajedničke. Grubi

poluprečnik se ne smanjuje.

Visina u `TelemetrySnapshot`-u mora biti visina iznad zemlje. Kokpit je puni iz barometra (relativno na tačku naoružavanja), a ne iz GPS visine koja je nadmorska i ranije je gurala pinove nekoliko puta predaleko.

4.8 Bindings: granica C++ / Python

`Bindings.cpp` (`PYBIND11_MODULE(DroneBackend, m)`) izlaže sve klase, strukture i enume. Vlastite logike nema, osim preimenovanja polja i nekoliko pomoćnih lambdi. Važna pravila:

- **Kopije, ne reference**: `get_latest_state()` vraća kopiju `DroneState`, pa Python ne može vidjeti napola upisano stanje i ne drži nikakvu bravu.
- **GIL**: blokirajući pozivi (`connect`, `start_auto`, `disconnect`, `auto_detect_fc`, JPEG pregled, zaustavljanje niti) imaju `py::call_guard<py::gil_scoped_release>`.
- **Životni vijek**: `CrsfLink.set_uplink(cmd)` ima `py::keep_alive<1,2>`, pa Python objekat `CommandLink`-a živi bar koliko i `CrsfLink` koji ga koristi iz C++ niti.
- **`to_dict()`** na `DroneState` daje običan rječnik, uključujući nizove fiksne dužine (motori, RC kanali) i ključeve `available_*` koji kažu šta trenutni izvor može dati.
- Komandni dio (`CommandValues`, `CommandLink`, `BridgeStatus`) je u `UplinkBindings.h` i registruje se sa `RegisterUplinkBindings(m)` prije `CrsfLink`-a.

5 Frontend (Python): aplikacija, radne niti i paneli

5.1 DroneCockpitApp: pokretanje, petlja, gašenje

Redoslijed pokretanja (`__init__`)

1. Kreiraju se `DroneLink` i `CrsfLink` (ili `None` ako ga stari `.pyd` nema, uz poruku u konzoli), postavlja se broj ćelija baterije, a zatim `CommandLink` , `radio.set_uplink(cmd_link)` i `UplinkController` .
2. Gradi se cijeli UI (`_setup_ui()`), **prije** bilo kakvog spajanja, da neuspjelo spajanje nikad ne ostavi pola interfejsa.
3. Kreira se `TelemetryWorker(hub, radio)` .
4. Kreira se `VideoLink` , pokreće `VideoWorker` i automatsko spajanje kamere. Video je nezavisan USB uređaj i ne čeka kontroler leta.
5. `fpv_view.attach(video_link)` predaje HWND FPV panela C++ niti; `set_telemetry_provider(_get_osd_telemetry)` povezuje OSD.
6. Kreira se i podešava `DetectionLink` (izvor frejmova, telemetrija, model, ulaz 960, interval 250 ms, FOV 100°, širine objekata, pragovi, CUDA FP16, popločavanje). Svaki noviji binding se poziva tek poslije `hasattr()` provjere, pa stari `.pyd` daje upozorenje umjesto pada aplikacije.
7. Pokreće se `DetectionWorker` . Prozor mape detekcija se gradi tek pri prvom otvaranju.
8. Telemetrija ne čeka USB kabl: `radio.start_auto()` , `TelemetryWorker.start()` , `UplinkController.start()` i pumpa kreću odmah, a `_auto_connect()` pokreće traženje kontrolera u niti `UsbProbe` . Instrumenti oživljavaju sa onog izvora koji se prvi javi.

Radni prostor sa panelima

Svaki instrument je u `DraggablePanel` -u na slobodnom `tk.Canvas` -u: pomjeranje za naslov, promjena veličine za ivice, desni klik za redoslijed. Ivice se lijepe za susjedne panele. Pokreti miša se ne primjenjuju na svaki događaj nego najviše svakih 33 ms (`_GEOM_APPLY_MS`) i tačno na otpuštanju. `LayoutStore` čuva imenovane rasporede u `config/cockpit_layouts.json` .

Petlja osvježavanja: `_update_loop()`

Jedan ponavljajući `root.after()` posao. Sljedeći krug se zakazuje 20 ms od **početka** trenutnog, ali nikad manje od 5 ms poslije njegovog kraja (`_reschedule_update()`), da Tk uvijek ima slobodno vrijeme za iscrtavanje. U svakom krugu:

1. osvježi status videa (nezavisno od telemetrije);
2. `_update_link_status()` : indikator radio veze, USB oznaka, rezultat `UsbProbe` -a, prekid USB-a i odluka o zamjeni lica panela;
3. `TelemetryWorker.get_frame()` (neblokirajuće);
4. podaci idu panelima, svakom svojom brzinom:

Panel	Svaki n-ti krug	Učestanost
ADI (`Drone3DView`), IMU, visina/VSI	2	25 Hz
Magnetometar / kurs i kuća	3	~17 Hz
FC status, GPS, tekst FPV statusa	5	10 Hz

Panel	Svaki n-ti krug	Učestanost
Provjere naoružavanja	10	5 Hz

Zapisi detekcija idu posebnim `after()` poslom (`_pump_detection_records()`), samo dok je prozor detekcija otvoren. Mjerenja su pokazala pad prosječnog troška kruga sa 18,8 ms na 2,9 ms i stvarnih 50 Hz umjesto ~24 Hz; `COCKPIT_PERF=1` uključuje mjerenje na stvarnom računaru (`core/ui_perf.py`).

Ponovno spajanje USB-a

- `_auto_connect()` pokreće jednu nit `UsbProbe` : `auto_detect_fc(exclude=[port radija])` , zatim `hub.connect(port)` i `radio.set_excluded_ports([port])` , da se dva skenera ne bore oko istog porta. Rezultat preuzima Tk nit; neuspjeh se ponavlja za 2 s.
- Ako je kabl spojen, ali veza nije zdrava 5 s (`USB_DROP_AFTER_S`), `_reconnect()` u niti `UsbDrop` radi `hub.disconnect()` i zakazuje novo traženje. Telemetrija u međuvremenu prelazi na ELRS ako je radio živ.

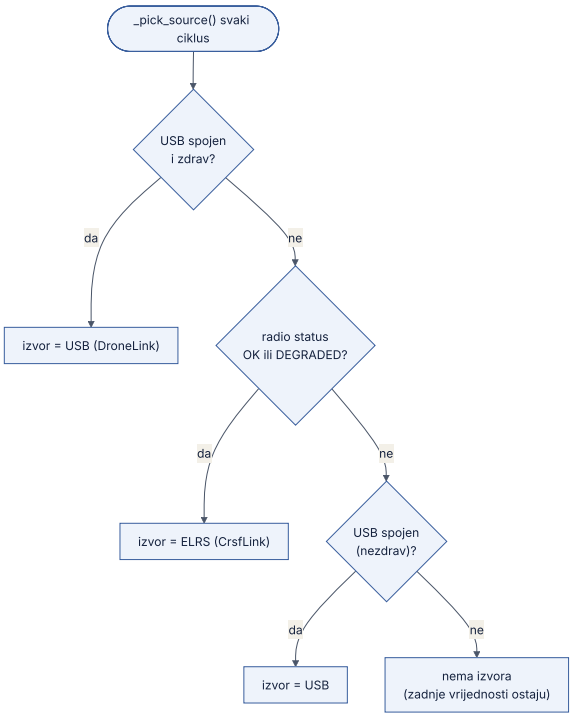
Gašenje (`shutdown()`)

Redoslijed je bitan: sačuva se raspored, otkažu `after()` poslovi, zatim se gasi po zavisnostima: `TelemetryWorker` , `hub.disconnect()` , `UplinkController.stop()` (neutralna komanda i 0,1 s da odu još koji neutralni okviri), `radio.set_uplink(None)` , `radio.disconnect()` , `VideoWorker` , `fpv_view.detach()` , `video_link.disconnect()` , `DetectionWorker` , `detection_link.stop()` i na kraju `root.destroy()` .

5.2 TelemetryWorker: izbor izvora i prevod podataka

Pravilo svih radnih niti: **nema `tkinter` importa i nema reference na widget**. Nit predaje običnu Python vrijednost, a Tk pumpa odlučuje kada je čita.

Izbor izvora: `_pick_source()`



Slika 6. Izbor izvora telemetrije u svakom ciklusu (60 Hz).

Stanje radija se vraća uvijek, i kad je USB aktivan, da indikator radio veze pokazuje stvarno stanje. `get_active_state()` radi isti izbor i koriste ga trampolini za OSD i detekciju, pa OSD, georeferenciranje i marker na mapi uvijek koriste **isti izvor koji prikazuju instrumenti**.

Jedino mjesto prevoda: `_build_ui_data(state, source)`

Ovo je jedino mjesto u frontend-u koje dira sirovi `DroneState`. Svi paneli koriste ravan rječnik koji ona vrati:

- roll i pitch se dijele sa 10 (MSP decistepeni); od yaw-a se oduzima pilotov trim;
- HDOP 9999 postaje 99,0 (oznaka „nevažeće"); lista satelita postaje lista rječnika;
- `rssi` 0–255 postaje `rc_link_quality` u procentima, sa −1 za „nema signala"; na ELRS-u to je stvarni uplink LQ;
- maska senzora se razbija u pojedinačne `sensor_*_present` logičke vrijednosti;
- dodaju se `link_source`, zastavice `available_*`, starosti `age_*_ms` i učestanosti `rate_*_hz`, a na ELRS-u i statistika veze (`elrs_*`);
- svako polje se čita sa `getattr(state, "polje", podrazumijevano)`, pa stariji `.pyd` daje razumnu vrijednost umjesto `AttributeError`-a koji bi srušio nit.

Status veze (`get_link_status()`) se gradi 10 puta u sekundi dok okviri teku i na svakom praznom krugu (0,1 s) kad izvora nema, a predaje se zamjenom cijelog rječnika.

5.3 VideoWorker i DetectionWorker

- `VideoWorker` (15 Hz) proziva samo `is_connected()`, `get_measured_fps()` i `get_device_name()`. **Nikad** ne poziva `get_latest_frame()`: video se crta u C++-u, a kopija frejma pod istom bravom koju koristi capture nit ne bi donijela ništa.

- **`DetectionWorker`** (4 Hz) čita jeftine brojače i `get_records_since(last_id)`, pa uvijek dobija samo nove zapise. `get_new_records()` vraća i prazni listu pod bravom. Indeks frejmova (`get_frame_index_since()`) se povlači isto inkrementalno, ograničen na 50 000 stavki (~3,5 h pri 4 Hz).

5.4 Prilagođavanje vezi: link_mode i dvoslojni paneli

Preko radija mnogi podaci ne stižu (sirovi IMU, motori, HDOP …). Da nule ne bi izgledale kao stvarna očitavanja, svi paneli prate tri pravila kroz `core/link_mode.py`:

1. **Ne stiže → sivi natpis** `USB` / `USB ONLY` (`available()`), bez alarma.
2. **Staro → „STALE"** (`stale_ms()`): stav stariji od 1,5 s, GPS, baterija, režim i baro stariji od 3 s. Zadnja vrijednost ostaje vidljiva, mijenja se samo boja.
3. **Oznaka izvora** `USB` / `RADIO` u zaglavlju panela.

Dvoslojni paneli (`core/dual_layer.py`): IMU i FC status imaju posebno „radio lice" (`RadioAttitudePanel`, `RadioFlightPanel`). `LayerSwitch` zamijeni lica tek kad novi izvor drži 1 s (`LAYER_HOLD_S`), da veza koja treperi ne bi stalno mijenjala prikaz. Pozicija, veličina i raspored ostaju isti. Skriveno lice je `grid_remove()`-ovano i ništa ne troši. Tajmer leta je jedan, u `FCStatusWidget`-u, i nastavlja da teče i dok je prikazano radio lice.

5.5 Instrumenti i prozori

Klasa	Uloga
`IMUWidget`	rotacija, G-sile, uglovi, odstupanje kursa, kašnjenje veze; trostepeni model upozorenja
`MagWidget`	kompasna ruža i kurs; strelica prema kući; isti widget za obje veze; kalibracija samo preko USB-a
`Drone3DView`	ADI (umjetni horizont) i kompasna traka; šrafiran kad je stav zastario
`BaroWidget`	traka visine i VSI; oznaka izvora visine
`GPSWidget`	mapa, navigacija, tabela satelita; mapa se crta jednom po idle ciklusu, pločice preuzima bazen od 4 niti
`FCStatusWidget`	naoružanje, režim, baterija, tajmer, senzori, CPU, motori, RC
`ArmingWidget`	lista provjera prije leta; provjera koja se ne može potvrditi preko radija daje sivu tačku i „READY — n CHECK(S) NEED USB"
`FPVWidget`	Tk okvir čiji HWND dobija `VideoLink`; prosljeđuje promjenu veličine, prikazuje status i FPS
`RadioLinkIndicator`	semafor radio veze u traci alata i oznaka aktivnog izvora
`DetectionMapWidget`	poseban `Toplevel` prozor: jedan red i jedan pin sa krugom nesigurnosti po objektu, filteri, pretraga scena po indeksu frejmova
`OsdSettingsPanel`, `OsdDragOverlay`	uključivanje i pozicioniranje OSD elemenata; čuvanje u `osd_layout.json`
`CommandPanel`	prozor „Laptop Control" za komande dronu (odjeljak 5.6)

5.6 Upravljanje sa računara: CommandPanel, UplinkController, AltitudeHold

`CommandPanel` (Tk nit)

- Tasteri W/S (naprijed/nazad), A/D (lijevo/desno), Q/E (yaw), R/F (gimbal), H (držanje visine), Space (stop). Prečice su vezane na prozor, a svi elementi imaju `takefocus=0`, pa Space ne može slučajno okinuti dugme.
- Kad prozor izgubi fokus (`_check_focus()`), svi pokreti se odmah zaustavljaju.
- Vizuelna povratna informacija: stanje tastature, pritisnuta dugmad, linija zahtjeva (žuto „BLOCKED" kad provjere ne prolaze), linija stvarno poslatog, kratke poruke događaja.
- Svakih 100 ms i na svaki taster šalje **otkucaj** `ui_heartbeat()`.

`UplinkController` (Python nit, 25 Hz)

U svakom krugu `_tick(dt, now)`:

1. pod bravom kopira intencije (pokret, brzine, gimbal, zahtjev za držanje visine);
2. ako otkucaja panela nema 0,5 s (`UI_TIMEOUT_S`), pokret se nulira; držanje visine i gimbal rade dalje;
3. provjere: uključeno, Lua most prisutan, ELRS telemetrija živa, dron naoružan, režim ANGLE (bench režim preskače zadnje dvije);
4. držanje visine: uključivanje bez skoka od gasa koji radio trenutno šalje (eho kanala 3), ispuštanje kad provjere padnu, prekidač se ugasi ili baro nestane duže od 1,5 s;
5. sastavlja `CommandValues` i poziva `set_command()`, što obnavlja lease;
6. objavljuje status (uključujući najveći period petlje u zadnje 2 s) i upisuje promjene u CSV log, koji se prazni na disk najviše jednom u sekundi.

`AltitudeHold`: regulator visine

Čista logika bez niti i Tk-a. PI regulator sa prigušenjem po vertikalnoj brzini:

```
e = cilj − visina
u = hover + Kp·e + I − Kd·vz        Kp = 40 µs/m, Ki = 8 µs/(m·s), Kd = 25 µs/(m/s)
I += Ki·e·dt                         ograničeno na ±60 µs, zamrznuto dok je izlaz u zasićenju
u ograničeno na hover ± 100 µs
```

- Cilj se mijenja najviše 0,5 m/s, pa je „gore 2 m" rampa.
- Vertikalna brzina dolazi iz vario telemetrije ili iz filtrirane derivacije visine (τ = 0,3 s).
- Visina starija od 800 ms zamrzava izlaz i ne integriše.
- Pojačanja su početne vrijednosti za probni stalak i podešavaju se testovima ALT-1 do ALT-3.

6 Tokovi podataka od kraja do kraja

6.1 Telemetrija preko USB kabla

1. `communicationLoop` šalje `$M<` upite, parseri pišu u `pending`, `commitState()` objavljuje (svakih 10 ms).
2. `TelemetryWorker` (60 Hz) uzima kopiju, `_build_ui_data()` pravi rječnik i stavlja ga u red dubine 3.
3. Tk pumpa (50 Hz) uzima najnoviji rječnik i daje ga panelima njihovom brzinom (5–25 Hz).

Najveće kašnjenje od kontrolera do ekrana je zbir: jedan MSP krug (≤ 10 ms), jedan krug radne niti (≤ 17 ms), jedan krug pumpe (≤ 20 ms) i period panela (40 ms za ADI). Procjena iz tih perioda: stav stiže na ekran za oko 50–90 ms.

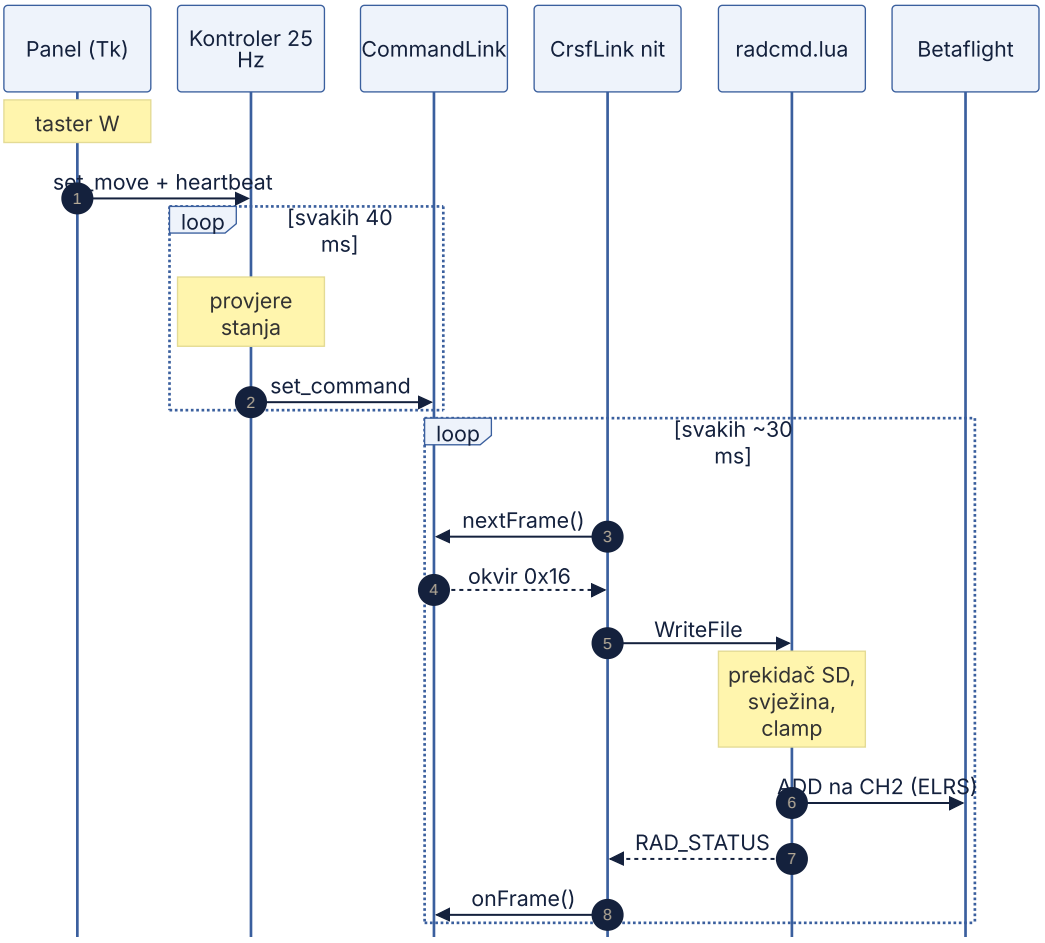
6.2 Telemetrija preko radija

1. Betaflight šalje CRSF okvire preko prijemnika i ELRS veze; `radcmd.lua` ih ponovo kodira i piše na USB.
2. `workerLoop` čita (≤ 20 ms), `StreamParser` slaže okvire, `CrsfStateMapper.apply()` ih upisuje u `DroneState`.
3. Od tog mjesta tok je isti kao preko USB-a. Učestanosti su manje: stav tipično nekoliko Hz, GPS i baterija ~1 Hz.

6.3 Od frejma do pina na mapi

1. `captureLoop` snimi frejm, sačuva ga kao `latestFrame_` i nacrta ga sa OSD-om u FPV prozor.
2. `inferenceLoop` (svakih ≥ 250 ms) uzme frejm i telemetriju, pokrene model, tracker i georeferenciranje (odjeljak 4.7) i napiše zapis.
3. `DetectionWorker` (4 Hz) povuče nove zapise; `_pump_detection_records()` ih preda `DetectionMapWidget`-u, koji ažurira red objekta i pin sa krugom.

6.4 Od tastera do Betaflight-a



Slika 7. Put komande „naprijed" od tastera W do kontrolera leta.

Kašnjenje: taster → kontroler ≤ 40 ms, kontroler → okvir ≤ 30 ms (plus jedan `ReadFile` do 20 ms), USB i skripta ~10 ms, ELRS paket nekoliko ms. Test UT-UPL-A3 traži medijanu ispod 100 ms od računara do kanala u Betaflight-u.

7 Otpornost na otkaze i sigurnost

7.1 Svaki zastoj završava neutralnim stanjem

Šta stane	Ko otkrije	Rok	Posljedica
Tk nit (dijalog, teško crtanje) dok je taster pritisnut	otkucaj panela u `UplinkController`-u	500 ms	pokret 0, držanje visine radi dalje
prozor komandi izgubi fokus	`_check_focus()`	odmah	svi pokreti zaustavljeni
nit `UplinkController`	lease u `CommandLink`-u	300 ms	palice i gas neutralni
C++ radna nit, USB kabl, cijela aplikacija	watchdog u `radcmd.lua`	500 ms	radio se vraća na pilotove palice
Lua most nestane	`bridgeTimeoutMs`	1,5 s	računar prestaje da piše
ELRS veza	failsafe Betaflight-a	po podešavanju FC-a	kako je podešeno na kontroleru

7.2 Otkazi izvora podataka

Događaj	Reakcija
USB kabl izvučen u letu	`DroneLink` postaje nezdrav poslije 5 neuspjelih krugova; `TelemetryWorker` prelazi na ELRS; paneli mijenjaju lice za ~1 s; poslije 5 s kokpit prekida i ponovo traži USB
Pocket izvučen	`CrsfLink` to vidi u roku od 1 s (`QueryDosDevice`), traži ga ponovo svakih 1 s; zadnje vrijednosti ostaju
dron izvan dometa	status `TELEMETRY_LOST` poslije 1,5 s; zadnja pozicija ostaje na mapi i kao marker drona
kamera izvučena ili prijemnik bez signala	`SignalLost` (ponovno spajanje, ubrzano na Windows obavještenje) ili `NoVideoInput` (upozorenje preko slike)
spor prolaz detekcije	video ne trpi (zasebne niti, niži prioritet, odmor po udjelu zauzetosti); popločavanje se samo gasi poslije 3 prespora prolaza
stari `DroneBackend.pyd`	`hasattr()` i `getattr()` provjere: upozorenje u konzoli i razumne podrazumijevane vrijednosti umjesto pada

7.3 Ovlašćenja pilota

Pilot uvijek ima konačnu riječ, i to na radiju, bez ikakvog softvera na računaru: prekidač SD gasi sve komande računara, pomaci se **dodaju** pilotovim palicama, gas računara je ograničen na ±100 µs oko pilotove palice, a naoružavanje, režimi leta i failsafe se nikad ne diraju sa računara.

8 Testiranje

Nivo	Gdje	Hardver	Šta pokriva
jedinični (Python)	`DroneCockpitUI/tests/unit`, `python -m pytest`	ne	link_mode (UT-LINK), dvoslojni paneli (UT-LAYER), mapa detekcija (UT-DETMAP), IMU widget, kontroler komandi i držanje visine, Lua most protiv C++ dekodera (Lua 5.2 sa lažnim EdgeTX API-jem); oko 90 testova
integracioni	`tests/integration` (IT-*)	FC na USB-u	IMU putanja kroz MSP (npr. IT-IMU-002 za skalu akcelerometra)
sistemski	`tests/system` (SAT-*)	FC na USB-u	prihvatni testovi IMU funkcija
C++	`DroneBackend/tests`	ne	`test_uplink.cpp` : protokol, CommandLink i test tri niti (666 provjera, i pod ThreadSanitizer-om); `tracker_sim.cpp` : simulacija praćenja; `detectionlink_run.cpp` : DetectionLink nad slikom
GoogleTest	`IMUTests/`	ne	IMU parser (lokalno)
na klupi	`tools/uplink_bench.py`, uputstvo	da	UT-UPL-A1…A5, S1…S4, ALT-1…3, MOV-1

9 Dodatak: konstante za podešavanje i pojmovnik

9.1 Konstante

Konstanta	Gdje	Vrijednost	Značenje
`MSP_BAUD`	DroneLink.h	57600	brzina USB veze sa kontrolerom
`POLL_INTERVAL_MS`	DroneLink.h	10	MSP prozivanje 100 Hz
`FAIL_THRESHOLD`	DroneLink.h	5	uzastopnih neuspjeha do nezdrave veze
`SV_POLL_TICKS` / `SLOW_POLL_TICKS`	DroneLink.h	100 / 50	sateliti ~1 s, baterija ~0,5 s
`lostTimeoutMs`	CrsfStateMapper.h	1500	bez FC okvira → TELEMETRY_LOST
`degradedLq` / `instrumentStaleMs`	CrsfStateMapper.h	70 % / 1500	prag za DEGRADED
`minSatsForFix` / `baroStaleMs`	CrsfStateMapper.h	5 / 2000	upotrebljiva pozicija; GPS rezerva za visinu
`Limits`	CommandLink.h	25 % / 30 % / 1150–1750 µs	granice palica, yaw-a i gasa
`leaseMs` / `sendPeriodMs` / `bridgeTimeoutMs`	CommandLink.h	300 / 30 / 1500	važenje komande, period slanja, nestanak mosta
`STALE_TICKS` , `MAX_STICK` , `THR_BAND`	radcmd.lua	50 / 300 / 200	watchdog 500 ms, ±30 % palice, ±100 µs gasa
`UPLINK_HZ` / `UI_TIMEOUT_S`	uplink_controller.py	25 / 0,5	ritam kontrolera, rok otkucaja panela
`UI_REFRESH_MS` / `UI_MIN_IDLE_MS`	DroneCockpitUI.py	20 / 5	pumpa 50 Hz, minimalno slobodno vrijeme
`POLL_HZ` / `_QUEUE_DEPTH`	telemetry_worker.py	60 / 3	ritam radne niti, dubina reda
`DETECTION_INPUT_SIZE` / `DETECTION_INTERVAL_MS`	DroneCockpitUI.py	960 / 250	ulaz modela, interval detekcije
`DETECTION_CONFIDENCE_THRESHOLD`	DroneCockpitUI.py	0,25	prag pouzdanosti (kalibrisan na FPV snimcima)
`DETECTION_CAMERA_FOV_DEG`	DroneCockpitUI.py	100	horizontalni FOV kamere

9.2 Pojmovnik

Pojam	Objašnjenje
MSP	MultiWii Serial Protocol; upit-odgovor protokol Betaflight-a preko serijske veze
CRSF	Crossfire protokol; okviri RC kanala i telemetrije koje koristi ExpressLRS

Pojam	Objašnjenje
ELRS	ExpressLRS, radio veza na 2,4 GHz između Pocket-a i prijemnika na dronu
USB-VCP	virtuelni COM port radio-predajnika; režim „Telem Mirror" samo šalje, režim „LUA" daje skripti oba smjera
LQ	Link Quality, procenat uspješno primljenih paketa
GIL	Global Interpreter Lock; u jednom trenutku samo jedna Python nit izvršava Python kod
lease	vremenski ograničeno važenje komande; bez obnove ističe
slew	ograničenje brzine promjene vrijednosti
letterbox	skaliranje slike u kvadrat uz čuvanje odnosa stranica i popunu ivica
NMS	Non-Maximum Suppression; od preklopljenih okvira zadržava najpouzdaniji
IoU	Intersection over Union; mjera preklapanja dva okvira
trampolin	kratka Python funkcija koju C++ nit poziva da dobije podatke (ovdje telemetriju)
bumpless	preuzimanje upravljanja bez skoka izlaza

Detaljnije stranice po modulima su u `docs/modules/` i `docs/frontend/`, a postavka radija i probe na klupi u `docs/modules/command-uplink-uputstvo.md`.